

Spring Framework – Day 13

Topic: Spring Bean Life Cycle

1. What is a Life Cycle?

In simple words, a life cycle means everything that happens to an object from the time it is born (created) to the time it dies (destroyed).

For example, look at a normal Java class:

```
class A {
    int a;

    A() {
    }

    public void m1() {
        System.out.println(a * a);
    }
}
```

Here, the object is created, some variables are initialized, some methods are called, and finally the object is destroyed. This journey - creation, initialization, use, and destruction - is called the life cycle of an object.

2. What is a Bean Life Cycle?

Q: In Core Java, who creates an object?

A: The JVM creates it, using the **new** keyword.

Q: In Spring, who creates the object?

A: The Spring IoC Container creates and manages the object.

Since Spring itself creates the object, Spring also controls:

- When the object should be created
- How the object should be initialized
- When the object should be destroyed

This complete journey of a bean, which is managed by the Spring container, is called the **Spring Bean Life Cycle**.

3. Why Do We Need Separate Life Cycle Methods?

Post-construction logic: Some operations need to be performed right after the object is created. This is called post-construction (or initialization) logic.

Pre-destruction logic: Some clean-up operations need to be performed just before the object is destroyed. This is called pre-destruction (or destruction) logic.

Look at this example:

```
class Rectangle {
    int length;
    int breadth;
```

```

    Rectangle(int length) {
        this.length = breadth;
        breadth = 0;
        area = length * breadth;
    }

    public void setBreadth(int breadth) {
        this.breadth = breadth;
    }

    public void cleanUp() {
        length = 0;
        breadth = 0;
    }
}

```

Not every value can be set at the time of object construction, and the constructor is not a good place to write clean-up logic either. This is why Spring provides separate, dedicated life cycle methods instead of forcing us to write this logic inside the constructor.

4. Spring Bean Life Cycle – Steps



Step 1: Container Starts

The IoC container starts working as soon as we create the `ApplicationContext`.

```
ApplicationContext context = new ClassPathXmlApplicationContext();
```

Step 2: Object Creation (Constructor)

Spring first creates the object by calling its constructor.

```

public Student() {
    System.out.println("Student Object Created");
}

// Output:
// Student Object Created

```

Step 3: Dependency Injection

Spring then injects the required values through setters.

```
<property name="name" value="Prashant"/>
```

Internally, Spring simply does:

```
student.setName("Prashant");
```

Step 4: Initialization

Suppose that after the object is created, we want to open a database connection, read some configuration, initialize a cache, or print a welcome message. Should this logic be written inside the constructor? No - because the constructor's only job is to create the object. Initialization logic should be kept separate.

For example, consider a class that opens a database connection:

```
class Operations {  
    public Operations() {  
        System.out.println("jdbc.oracle.driver.OracleDriver");  
        System.out.println("jdbc:oracle:thin:@localhost:1521:xe");  
        System.out.println("system");  
        System.out.println("manager");  
    }  
}
```

If we create the object of this class multiple times, the constructor will also run multiple times. As a result, the database connection logic will run every single time, which is not needed, since the connection should ideally be made only once. This is exactly why Spring provides a special method for this purpose: the **init-method**.

Step 5: Bean Ready

At this point, the bean is completely prepared, and the application starts using it.

Step 8: Destroy

When the container is closed, Spring automatically calls the **destroy-method**. This is used for:

- Closing a database connection
- Closing a file
- Releasing memory
- Closing a socket

5. Proof of Concept – Example 1

Bean Class: A.java

```
package com.day13;

public class A {

    int a1;

    public A() {
        System.out.println("Constructor executed");
    }

    public int getA1() {
        return a1;
    }

    public void setA1(int a1) {
        System.out.println("Setter method executes");
        this.a1 = a1;
    }

    public void init() {
        System.out.println("One time configuration done");
    }

    public void destroy() {
        System.out.println("All resources closed");
    }

    public void display() {
        System.out.println("Custom method executes");
    }

    @Override
    public String toString() {
        return "A [a1=" + a1 + "]";
    }
}
```

Configuration File: application-context.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="a" class="com.day13.A"
          init-method="init"
          destroy-method="destroy">
        <property name="a1" value="100"></property>
    </bean>

</beans>
```

Main Class

```
public class Main {
    public static void main(String[] args) {
        ClassPathXmlApplicationContext context =
```

```

        new ClassPathXmlApplicationContext(
            "com/day13/application-context.xml");

        A bean = context.getBean("a", A.class);

        System.out.println(bean);

        context.close();
    }
}

```

Note: In this example, the `init`-method and `destroy`-method attributes tell Spring which method to call after object creation (initialization) and which method to call before the bean is destroyed (clean-up), respectively.

6. Example 2 – Life Cycle with Message Converters

This example shows the same life cycle concept applied to a more realistic scenario, where one bean depends on another bean (dependency injection) along with `init` and `destroy` methods.

Interface: IMessageConveter.java

```

package com.day13;

public interface IMessageConveter {
    public String convertTo(String message);
}

```

Implementation 1: HtmlMessageConverter.java

```

public class HtmlMessageConverter implements IMessageConveter {

    public HtmlMessageConverter() {
    }

    public String convertTo(String message) {
        return "<html>" + message + "</html>";
    }
}

```

Implementation 2: PDFMessageConverter.java

```

public class PDFMessageConverter implements IMessageConveter {

    public PDFMessageConverter() {
    }

    public String convertTo(String message) {
        return "<pdf>" + message + "</pdf>";
    }
}

```

Main Bean: MessageConverter.java

```

public class MessageConverter {
    IMessageConveter converter;
    String message;

    public MessageConverter() {
        System.out.println("Message Converter object created");
    }

    public void convert(String message) {
        String result = converter.convertTo(message);
    }
}

```

```

        System.out.println(result);
    }

    public IMessageConveter getConverter() {
        return converter;
    }

    public void setConverter(IMessageConveter converter) {
        System.out.println("setter executed for converter");
        this.converter = converter;
    }

    public String getMessage() {
        return message;
    }

    public void setMessage(String message) {
        System.out.println("Setter executed for message");
        this.message = message;
    }

    public void init() {
        System.out.println("Init method called");
        this.convert(message);
    }

    public void destroy() {
        System.out.println("Destruction method called");
        converter = null;
        message = null;
    }
}

```

Configuration File: application-context.xml

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="html" class="com.day12.HtmlMessageConverter">
    </bean>

    <bean id="pdf" class="com.day12.PDFMessageConverter">
    </bean>

    <bean id="messageConverter" class="com.day12.MessageConverter"
        init-method="init"
        destroy-method="destroy">
        <property name="converter" ref="html"></property>
        <property name="message" value="Hello guys"></property>
    </bean>

</beans>

```

Main Class: MyApp.java

```

public class MyApp {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        ClassPathXmlApplicationContext context =

```

```
        new ClassPathXmlApplicationContext(  
            "com/day12/application-context.xml");  
    }  
}
```

Note: Here, the converter bean (html or pdf) is injected into MessageConverter through the property tag with ref, before the init-method runs. That is why, inside init(), the converter is already available and convert(message) can be called safely.

7. Quick Summary

- Spring IoC Container creates and manages the bean, not the JVM directly.
- Life cycle order: Container Starts -> Constructor -> Dependency Injection -> Initialization (init-method) -> Bean Ready -> Application Running -> Container Shutdown -> Destroy (destroy-method).
- init-method is used for one-time setup work (like opening a connection).
- destroy-method is used for clean-up work (like closing a connection).
- Constructor should only be used to create the object, not to write business or setup logic.